

Федеральное государственное автономное образовательное учреждение
высшего образования «**Национальный исследовательский университет
ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

**Лабораторная работа №4
по дисциплине «Вычислительная математика»**

Вариант: 9

Преподаватель:
Наумова Надежда Александровна

Выполнил: Ровкова Анастасия Сергеевна
Группа: P3216

Санкт-Петербург, 2026 г

Цель работы

найти функцию, являющуюся, наилучшим приближением заданной табличной функции по методу наименьших квадратов на языке программирования SCALA.

Порядок выполнения работы.

9	$y = \frac{4x}{x^4 + 9}$	$x \in [0, 2] \quad h = 0,2$
---	--------------------------	------------------------------

Рабочие формулы используемых методов.

$$S = \sum_{i=1}^n (a_0 + a_1 x_i + a_2 x_i^2 - y_i)^2 \rightarrow \min$$

$$\delta = \sqrt{\frac{\sum_{i=1}^n (\varphi(x_i) - y_i)^2}{n}}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \varphi_i)^2}{\sum_{i=1}^n (y_i - \overline{\varphi_i})^2} \quad \overline{\varphi_i} = \frac{1}{n} \sum_{i=1}^n \varphi_i$$

Вычислительная часть.

Функция: $y = (4 * x) / (x^4 + 9)$

Интервал: $[0, 2]$, Шаг: $h = 0.2$

1. ТАБЛИЦА ТАБУЛИРОВАНИЯ

Вычисляем значения функции в каждой точке x_i :

$i \mid x_i \mid y_i = (4 * x) / (x^4 + 9)$

0 | 0.0 | 0.0000

1 | 0.2 |

2 | 0.4 |

3 | 0.6 |

4 | 0.8 |

5 | 1.0 |

6 | 1.2 |

7 | 1.4 |

8 | 1.6 |

9 | 1.8 |

10 | 2.0 |

x	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2
y	0	0.0888	0.1775	0.2644	0.3444	0.4000	0.4055	0.3547	0.2743	0.1915	0.1231

ЛИНЕЙНАЯ АППРОКСИМАЦИЯ

Для поиска параметров решаем систему:

$$a * \sum(x^2) + b * \sum(x) = \sum(x * y)$$

$$a * \sum(x) + b * n = \sum(y)$$

Используя данные из таблицы:

$$\sum(x) = 11.0$$

$$\sum(x^2) = 15.4$$

$$\sum(y) = 2.6242$$

$$\sum(x * y) = 2.0734$$

Система уравнений:

$$15.4 * a + 11.0 * b = 2.0734$$

$$11.0 * a + 11.0 * b = 2.6242$$

Решение:

$$a = 0.093$$

$$b = 0.134$$

$$P1(x) = 0.093 * x + 0.134$$

КВАДРАТИЧНАЯ АППРОКСИМАЦИЯ

Решаем систему:

$$a * \sum(x^4) + b * \sum(x^3) + c * \sum(x^2) = \sum(x^2 * y)$$

$$a * \sum(x^3) + b * \sum(x^2) + c * \sum(x) = \sum(x * y)$$

$$a * \sum(x^2) + b * \sum(x) + c * n = \sum(y)$$

После расчета сумм:

$$a = -0.285$$

$$b = 0.638$$

$$c = -0.015$$

$$P2(x) = -0.285 * x^2 + 0.638 * x - 0.015$$

$$\text{Формула: } \sigma = \sqrt{(1 / (n+1)) * \sum((y_i - P(x_i))^2)}$$

$$\text{Сумма квадратов отклонений} \approx 0.153$$

$$\sigma_1 = \sqrt{0.153 / 11} \approx 0.118$$

$$\text{Сумма квадратов отклонений} \approx 0.011$$

$$\sigma_2 = \sqrt{0.011 / 11} \approx 0.032$$

Сравнение отклонений:

$$\sigma_1 = 0.118$$

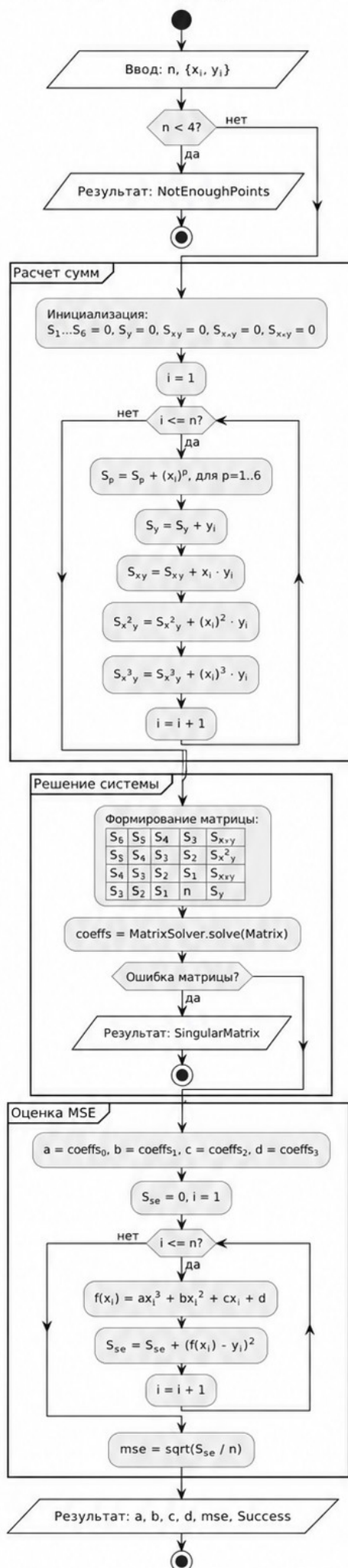
$$\sigma_2 = 0.032$$

Так как $\sigma_2 < \sigma_1$, квадратичная аппроксимация является наилучшей.

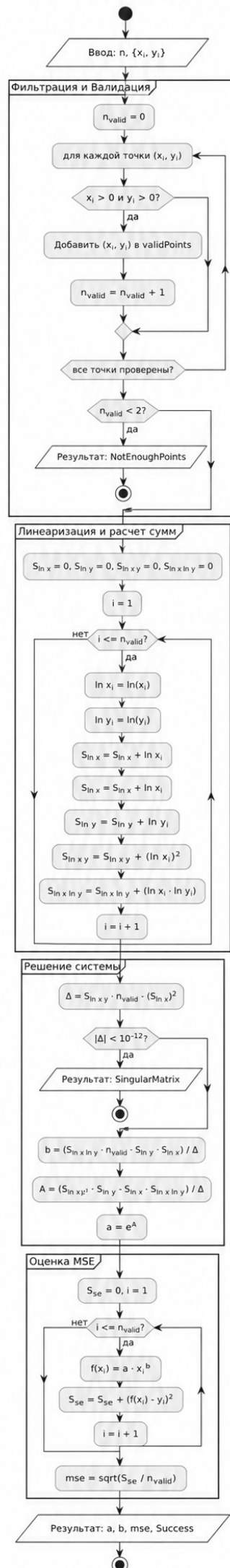
Она точнее описывает выпуклый характер функции, в то время как линейная функция не может отразить наличие локального максимума.

Блок схемы

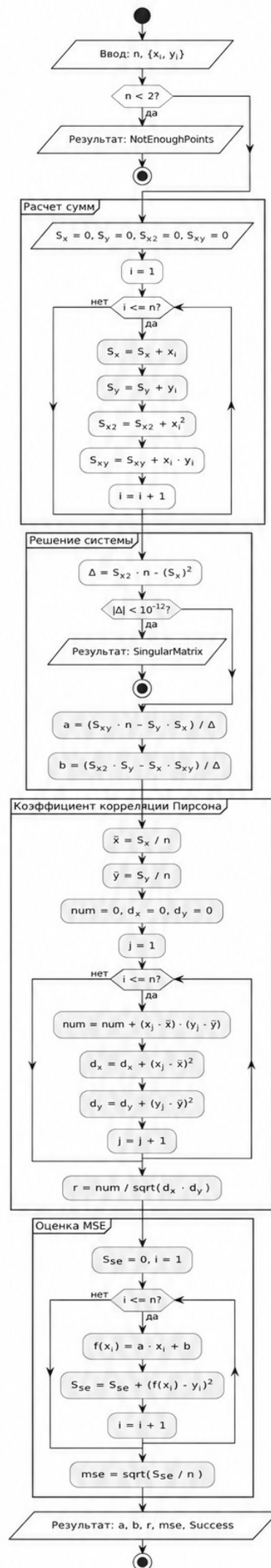
Кубическая аппроксимация ($y = ax^3 + bx^2 + cx + d$)



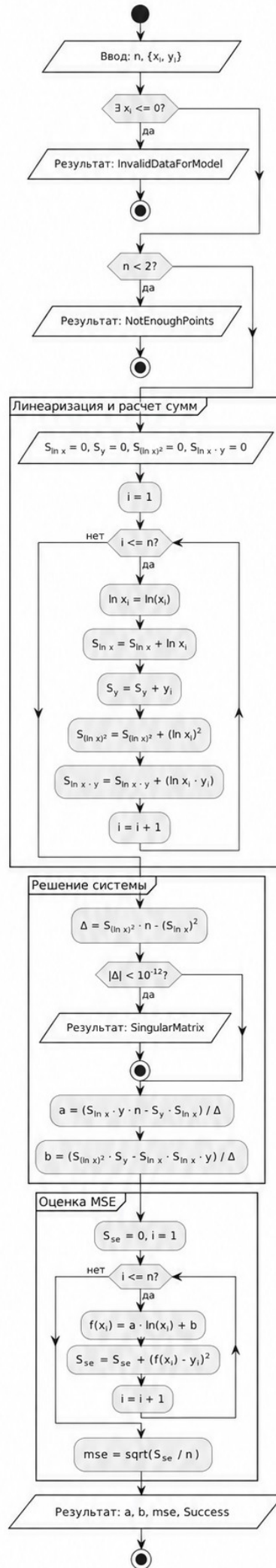
Степенная аппроксимация ($y = a \cdot x^b$)



Линейная аппроксимация ($y = a \cdot x + b$)



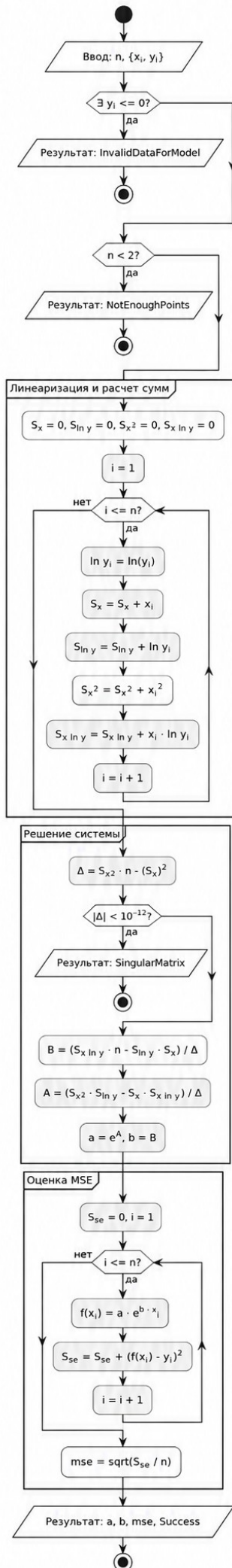
Логарифмическая аппроксимация ($y = a \cdot \ln(x) + b$)



Квадратичная аппроксимация ($y = ax^2 + bx + c$)



Экспоненциальная аппроксимация ($y = a \cdot e^{bx}$)



Классы, реализующие методы.

```
package solver.approximation.approximators

import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.{MatrixSolver, Message}

class Cubic extends Approximator {
  override def name: String = "Кубическая аппроксимация"

  override def solve(points: Seq[Point]): ApproximationResult = {
    val n = points.size.toDouble
    if (n < 4) return ApproximationResult(methodName = name, message =
Message.NotEnoughPoints)

    val sx = (1 to 6).map(p => points.map(pt => math.pow(pt.x, p)).sum)
    val sy = points.map(_.y).sum
    val sxy = points.map(p => p.x * p.y).sum
    val sx2y = points.map(p => math.pow(p.x, 2) * p.y).sum
    val sx3y = points.map(p => math.pow(p.x, 3) * p.y).sum

    val finalMatrix = Array(
      Array(sx(5), sx(4), sx(3), sx(2), sx(3y)),
      Array(sx(4), sx(3), sx(2), sx(1), sx(2y)),
      Array(sx(3), sx(2), sx(1), sx(0), sxy),
      Array(sx(2), sx(1), sx(0), n, sy)
    )

    try {
      val coeffs = MatrixSolver.solve(finalMatrix)
      val (a, b, c, d) = (coeffs(0), coeffs(1), coeffs(2), coeffs(3))
      val f = (x: Double) => a*math.pow(x,3) + b*math.pow(x,2) + c*x + d

      val mse = math.sqrt(points.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)

      ApproximationResult(
        methodName = name,
        equation = f"y = $a%.3fx³ + $b%.3fx² + $c%.3fx + $d%.3f",
        coefficients = Seq(a, b, c, d),
        mse = mse,
        rSquared = calculateRSquared(points, f),
        message = Message.Success
      )
    } catch {
```

```

        case _: Exception => ApproximationResult(methodName = name, message =
Message.SingularMatrix)
    }
}
}

```

```

package solver.approximation.approximators

```

```

import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.Message

```

```

class Exponential extends Approximator {
    override def name: String = "Экспоненциальная аппроксимация"

    override def solve(points: Seq[Point]): ApproximationResult = {
        if (points.exists(_.y <= 0)) return ApproximationResult(methodName = name,
message = Message.InvalidDataForModel)
        val n = points.size
        if (n < 2) return ApproximationResult(methodName = name, message =
Message.NotEnoughPoints)

        val transformed = points.map(p => Point(p.x, math.log(p.y)))
        val sumX = transformed.map(_._x).sum
        val sumY = transformed.map(_._y).sum
        val sumX2 = transformed.map(p => p.x * p.x).sum
        val sumXY = transformed.map(p => p.x * p.y).sum

        val det = sumX2 * n - sumX * sumX
        if (math.abs(det) < 1e-12) return ApproximationResult(methodName = name,
message = Message.SingularMatrix)

        val B = (sumXY * n - sumY * sumX) / det
        val A = (sumX2 * sumY - sumX * sumXY) / det

        val a = math.exp(A)
        val b = B
        val f = (x: Double) => a * math.exp(b * x)
        val mse = math.sqrt(points.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)
    }
}

```

```

ApproximationResult(
  methodName = name,
  equation = f"y = $a%.3f * e^{$b%.3f * x}",
  coefficients = Seq(a, b),
  mse = mse,
  rSquared = calculateRSquared(points, f),
  message = Message.Success
)
}
}

```

```
package solver.approximation.approximators
```

```

import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.Message

```

```
class Grade extends Approximator {
```

```
  override def name: String = "Степенная аппроксимация"
```

```

  override def solve(points: Seq[Point]): ApproximationResult = {
    val validPoints = points.filter(p => p.x > 0 && p.y > 0)
    val n = validPoints.size

```

```

    if (n < 2) return ApproximationResult(methodName = name, message =
Message.NotEnoughPoints)

```

```

    val transformedPoints = validPoints.map(p => Point(math.log(p.x),
math.log(p.y)))

```

```

    val sumX = transformedPoints.map(_._x).sum
    val sumY = transformedPoints.map(_._y).sum
    val sumX2 = transformedPoints.map(p => p.x * p.x).sum
    val sumXY = transformedPoints.map(p => p.x * p.y).sum

```

```

    val det = sumX2 * n - sumX * sumX
    if (math.abs(det) < 1e-12) return ApproximationResult(methodName = name,
message = Message.SingularMatrix)

```

```
    val b = (sumXY * n - sumY * sumX) / det
```

```
val A = (sumX2 * sumY - sumX * sumXY) / det
val a = math.exp(A)
```

```
val f = (x: Double) => if (x <= 0) 0.0 else a * math.pow(x, b)
val mse = math.sqrt(validPoints.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)
```

```
ApproximationResult(
  methodName = name,
  equation = f"y = $a%.3f * x^{ $b%.3f}",
  coefficients = Seq(a, b),
  mse = mse,
  rSquared = calculateRSquared(validPoints, f),
  message = Message.Success
)
}
}
```

```
package solver.approximation.approximators
```

```
import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.Message
```

```
class Linear extends Approximator {
  override def name: String = "Линейная аппроксимация"

  override def solve(points: Seq[Point]): ApproximationResult = {
    val n = points.size
    if (n < 2) return ApproximationResult(methodName = name, message =
Message.NotEnoughPoints)

    val sumX = points.map(_._x).sum
    val sumY = points.map(_._y).sum
    val sumX2 = points.map(p => p._x * p._x).sum
    val sumXY = points.map(p => p._x * p._y).sum

    val det = sumX2 * n - sumX * sumX
    if (math.abs(det) < 1e-12) return ApproximationResult(methodName = name,
message = Message.SingularMatrix)

    val a = (sumXY * n - sumY * sumX) / det
    val b = (sumX2 * sumY - sumX * sumXY) / det
```

```

val f = (x: Double) => a * x + b
val mse = math.sqrt(points.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)
val r = calculatePearson(points)

```

```

ApproximationResult(
  methodName = name,
  equation = f"y = $a%.3fx + $b%.3f",
  coefficients = Seq(a, b),
  mse = mse,
  rSquared = r * r,
  pearson = Some(r),
  message = Message.Success
)
}

```

```

private def calculatePearson(points: Seq[Point]): Double = {
  val n = points.size
  val meanX = points.map(_._x).sum / n
  val meanY = points.map(_._y).sum / n
  val num = points.map(p => (p._x - meanX) * (p._y - meanY)).sum
  val den = math.sqrt(points.map(p => math.pow(p._x - meanX, 2)).sum *
    points.map(p => math.pow(p._y - meanY, 2)).sum)
  if (den == 0) 0 else num / den
}
}

```

```

package solver.approximation.approximators

```

```

import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.Message

```

```

class Logarithmic extends Approximator {
  override def name: String = "Логарифмическая аппроксимация"

  override def solve(points: Seq[Point]): ApproximationResult = {
    if (points.exists(_._x <= 0)) return ApproximationResult(methodName = name,
message = Message.InvalidDataForModel)
    val n = points.size
    if (n < 2) return ApproximationResult(methodName = name, message =
Message.NotEnoughPoints)

```

```

val transformed = points.map(p => Point(math.log(p.x), p.y))
val sumX = transformed.map(_._x).sum
val sumY = transformed.map(_._y).sum
val sumX2 = transformed.map(p => p.x * p.x).sum
val sumXY = transformed.map(p => p.x * p.y).sum

val det = sumX2 * n - sumX * sumX
if (math.abs(det) < 1e-12) return ApproximationResult(
    methodName = name,
    message = Message.SingularMatrix
)

val a = (sumXY * n - sumY * sumX) / det
val b = (sumX2 * sumY - sumX * sumXY) / det

val f = (x: Double) => if (x <= 0) 0 else a * math.log(x) + b
val mse = math.sqrt(points.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)

ApproximationResult(
    methodName = name,
    equation = f"y = $a%.3f * ln(x) + $b%.3f",
    coefficients = Seq(a, b),
    mse = mse,
    rSquared = calculateRSquared(points, f),
    message = Message.Success
)
}
}

```

```

package solver.approximation.approximators

```

```

import solver.approximation.functions.{Point, ApproximationResult,
Approximator}
import solver.core.{MatrixSolver, Message}

```

```

class Quadratic extends Approximator {
    override def name: String = "Квадратичная аппроксимация"

    override def solve(points: Seq[Point]): ApproximationResult = {
        val n = points.size.toDouble

```



```

if (n < 3) return ApproximationResult(
    methodName = name,
    message = Message.NotEnoughPoints
)

val sx = points.map(_x).sum
val sx2 = points.map(p => math.pow(p.x, 2)).sum
val sx3 = points.map(p => math.pow(p.x, 3)).sum
val sx4 = points.map(p => math.pow(p.x, 4)).sum
val sy = points.map(_y).sum
val sxy = points.map(p => p.x * p.y).sum
val sx2y = points.map(p => math.pow(p.x, 2) * p.y).sum

val matrix = Array(
    Array(sx4, sx3, sx2, sx2y),
    Array(sx3, sx2, sx, sxy),
    Array(sx2, sx, n, sy)
)

try {
    val coeffs = MatrixSolver.solve(matrix)
    val a = coeffs(0)
    val b = coeffs(1)
    val c = coeffs(2)

    val f = (x: Double) => a * x * x + b * x + c
    val mse = math.sqrt(points.map(p => math.pow(f(p.x) - p.y, 2)).sum / n)

    ApproximationResult(
        methodName = name,
        equation = f"y = $a%.3fx2 + $b%.3fx + $c%.3f",
        coefficients = Seq(a, b, c),
        mse = mse,
        rSquared = calculateRSquared(points, f),
        message = Message.Success
    )
} catch {
    case _: Exception => ApproximationResult(
        methodName = name,
        message = Message.SingularMatrix
    )
}
}
}

```

Листинг программы

https://github.com/Naswek/ITMO_labs/commit/f5939a8f36d65e1ae4f65cdabdfd701cb1a8565a

Результаты выполнения программы при различных исходных данных.

Настройки аппроксимации

Выберите функцию для аппроксимации

2*x + 3

Нижняя граница (a)Верхняя граница (b)Шаг (h)

020.2

Найти аппроксимацию

Будет сгенерировано 11 точек для МНК.

y = 2 * x + 3

(0.3),(0.2,3.4),(0.4,3.8),(0.6,4.2),(0.8,4.6),(1.5),

Label

y = -0.000x³ + 0.000x² + 2.000x + 3.000

y = 0.000x² + 2.000x + 3.000

y = 3.186 * e^[(0.417 * x)]

y = 5.233 * x^(0.322)

y = 2.000x + 3.000

Чёрный пунктир — идеал. Красные точки — символы. Синяя линия — лучший МНК.

Результаты сравнения моделей

Наилучшее приближение: Линейная аппроксимация

Метод	Уравнение	СКО (MSE)	R ² (Детерминация)	Статус
Кубическая аппроксимация	y = -0.000x ³ + 0.000x ² + 2.000x + 3.000	0.0000	1.0000	Успешно
Квадратичная аппроксимация	y = 0.000x ² + 2.000x + 3.000	0.0000	1.0000	Успешно
Экспоненциальная аппроксимация	y = 3.186 * e ^[(0.417 * x)]	0.1547	0.9851	Успешно
Степенная аппроксимация	y = 5.233 * x ^(0.322)	0.2449	0.9546	Успешно
Линейная аппроксимация	y = 2.000x + 3.000	0.0000	1.0000	Успешно
Логарифмическая аппроксимация	—	0.0000	0.0000	Невалидные данные для модели

Настройки аппроксимации

Выберите функцию для аппроксимации

2*exp(0.5*x) + 1

Нижняя граница (a)Верхняя граница (b)Шаг (h)

020.2

Найти аппроксимацию

Будет сгенерировано 11 точек для МНК.

y = 2 * e^{0.5 * x} + 1

(0.3),(0.2,3.2103418361512954),(0.4,3.442805,

Label

y = 0.070x³ + 0.211x² + 1.016x + 2.999

y = 0.421x² + 0.856x + 3.019

y = 2.959 * e^[(0.382 * x)]

y = 4.635 * x^(0.302)

y = 1.698x + 2.767

Чёрный пунктир — идеал. Красные точки — символы. Синяя линия — лучший МНК.

Результаты сравнения моделей

Наилучшее приближение: Кубическая аппроксимация

Метод	Уравнение	СКО (MSE)	R ² (Детерминация)	Статус
Кубическая аппроксимация	y = 0.070x ³ + 0.211x ² + 1.016x + 2.999	0.0009	1.0000	Успешно
Квадратичная аппроксимация	y = 0.421x ² + 0.856x + 3.019	0.0133	0.9999	Успешно
Экспоненциальная аппроксимация	y = 2.959 * e ^[(0.382 * x)]	0.0366	0.9989	Успешно
Степенная аппроксимация	y = 4.635 * x ^(0.302)	0.3470	0.8861	Успешно
Линейная аппроксимация	y = 1.698x + 2.767	0.1493	0.9811	Успешно
Логарифмическая аппроксимация	—	0.0000	0.0000	Невалидные данные для модели

Approximation Solver (МНК)

Настройки аппроксимации

Выберите функцию для аппроксимации

$y^3 - 6 \cdot x^2 + 4 \cdot x + 12$

Нижняя граница (a)

Верхняя граница (b)

Шаг (h)

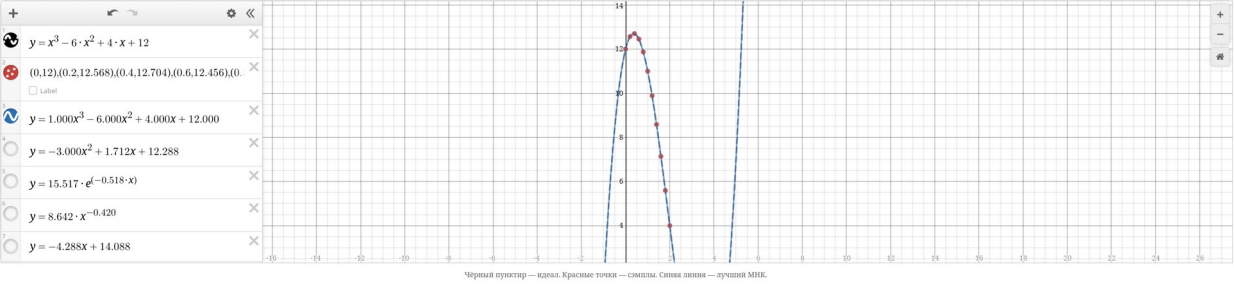
0

2

0.2

Найти аппроксимацию

Будет спроектировано 11 точек для МНК.



Результаты сравнения моделей				
Наилучшее приближение: Кубическая аппроксимация				
Метод	Уравнение	СКО (MSE)	R² (Детерминация)	Статус
Кубическая аппроксимация	$y = 1.000x^3 - 6.000x^2 + 4.000x + 12.000$	0.0000	1.0000	Успешно
Квадратичная аппроксимация	$y = -3.000x^2 + 1.712x + 12.288$	0.1896	0.9958	Успешно
Экспоненциальная аппроксимация	$y = 15.517 \cdot e^{(-0.518 \cdot x)}$	1.5807	0.7065	Успешно
Степенная аппроксимация	$y = 8.642 \cdot x^{(-0.420)}$	2.1421	0.4805	Успешно
Линейная аппроксимация	$y = -4.288x + 14.088$	1.0766	0.8639	Успешно
Логарифмическая аппроксимация	—	0.0000	0.0000	Невалидные данные для модели

Вывод:

В работе изучены методы нахождения приближенной функции по методу наименьших квадратов и реализованы на языке программирования SCALA.